

Database Management System (DBMS)

BAB 9 — Indexing dan Query Processing

1. Pengertian Index

Index adalah struktur data tambahan yang digunakan untuk mempercepat proses pencarian data pada database.

Tanpa index:

Database harus memeriksa data satu per satu

Dengan index:

Database dapat langsung menuju lokasi data

2. Analogi Index

Sama seperti daftar isi pada buku.

Tanpa daftar isi:

Mencari halaman satu per satu

Dengan daftar isi:

Langsung menuju halaman yang dicari

3. Tujuan Index

Mempercepat pencarian data

Mempercepat sorting

Mempercepat JOIN

Mengurangi waktu eksekusi query

4. Kekurangan Index

Index juga memiliki biaya.

Contoh:

Membutuhkan storage tambahan

INSERT lebih lambat

UPDATE lebih lambat

DELETE lebih lambat

5. Cara Kerja Index

Ketika query dijalankan:

```
SELECT *  
FROM mahasiswa  
WHERE nim='101';
```

DBMS dapat menggunakan index untuk menemukan data lebih cepat.

6. Full Table Scan

Jika tidak ada index:

DBMS membaca seluruh tabel

Proses ini disebut:

Full Table Scan

7. Search Key

Search Key adalah atribut yang digunakan dalam index.

Contoh:

NIM

Email

KodeProduk

8. Primary Index

Index yang dibuat berdasarkan Primary Key.

Contoh:

PRIMARY KEY(nim)

Biasanya otomatis dibuat oleh DBMS.

9. Secondary Index

Index yang dibuat pada atribut selain Primary Key.

Contoh:

```
CREATE INDEX idx_nama  
ON mahasiswa(nama);
```

10. Clustered Index

Data fisik tabel disimpan mengikuti urutan index.

Karakteristik:

Hanya satu Clustered Index
dalam satu tabel

11. Non-Clustered Index

Index terpisah dari data fisik.

Karakteristik:

Dapat memiliki banyak index

12. Dense Index

Setiap record memiliki entry pada index.

Contoh:

101 → Baris 1

102 → Baris 2

103 → Baris 3

13. Sparse Index

Tidak semua record memiliki entry.

Biasanya hanya sebagian record tertentu.

14. Single-Level Index

Index hanya memiliki satu level.

Cocok untuk:

Data kecil

15. Multi-Level Index

Index memiliki beberapa level.

Cocok untuk:

Database besar

16. B-Tree

Struktur index yang paling umum digunakan.

Karakteristik:

Terurut

Seimbang

Pencarian cepat

17. Keuntungan B-Tree

Search cepat

Insert cepat

Delete cepat

18. B+ Tree

Pengembangan dari B-Tree.

Perbedaan utama:

Seluruh data berada di leaf node

19. Mengapa B+ Tree Populer

Karena:

Lebih efisien

Range query lebih cepat

20. Hash Index

Menggunakan fungsi hash.

Contoh:

ID → Hash Function → Lokasi Data

21. Keunggulan Hash Index

Sangat cepat untuk:

Pencarian exact match

Contoh:

WHERE id = 100

22. Kelemahan Hash Index

Kurang cocok untuk:

>

<

BETWEEN

23. Query Processing

Query Processing adalah proses DBMS menerjemahkan query menjadi operasi yang dapat dieksekusi.

24. Tahapan Query Processing

Parsing

Optimization

Execution

25. Parsing

DBMS memeriksa:

Sintaks

Struktur query

Nama tabel

Nama kolom

26. Query Optimization

DBMS mencari cara paling efisien untuk menjalankan query.

Contoh:

Memilih index terbaik

Menentukan urutan JOIN

27. Query Execution

DBMS menjalankan query yang telah dioptimasi.

28. Query Optimizer

Komponen yang menentukan strategi terbaik untuk menjalankan query.

Tujuan:

Meminimalkan biaya eksekusi

29. Execution Plan

Rencana eksekusi query yang dibuat optimizer.

Berisi:

Scan

Join

Sort

Index Usage

30. Cost-Based Optimization

Optimizer menghitung biaya berbagai alternatif.

Kemudian memilih:

Biaya terkecil

31. Selectivity

Mengukur seberapa efektif suatu kondisi query.

Contoh:

```
WHERE nim='101'
```

Lebih selektif dibanding:

```
WHERE jenis_kelamin='L'
```

32. Query dengan Index

```
SELECT *  
FROM mahasiswa  
WHERE nim='101';
```

Biasanya menggunakan:

Index Scan

33. Query Tanpa Index

```
SELECT *  
FROM mahasiswa  
WHERE nama='Andi';
```

Jika nama tidak di-index:

Full Table Scan

34. Join Processing

JOIN sering menjadi operasi paling mahal.

Contoh:

Mahasiswa

KRS

Mata Kuliah

digabungkan dalam satu query.

35. Nested Loop Join

Cara sederhana melakukan JOIN.

Konsep:

Setiap baris tabel pertama

dibandingkan dengan

setiap baris tabel kedua

36. Sort Merge Join

Kedua tabel diurutkan terlebih dahulu.

Kemudian:

Digabungkan

37. Hash Join

Menggunakan struktur hash untuk mempercepat JOIN.

Cocok untuk:

Data besar

38. Optimasi Query

Beberapa teknik:

Gunakan index

Hindari SELECT *

Gunakan WHERE

Batasi JOIN yang tidak perlu

39. Kesalahan Umum

Membuat Terlalu Banyak Index

Akibat:

INSERT lebih lambat

UPDATE lebih lambat

Tidak Membuat Index

Akibat:

Query pencarian lambat

Menggunakan SELECT *

Akibat:

Data yang tidak diperlukan ikut dibaca

Mengabaikan Execution Plan

Dapat menyebabkan query tidak optimal.

40. Hubungan Index dan Query Processing

Index membantu optimizer

Optimizer memilih execution plan

Execution plan menentukan performa query

Soal Bab 9

Soal 1

Jelaskan pengertian index dan manfaatnya dalam database.

Soal 2

Bandingkan:

Primary Index

Secondary Index

Soal 3

Jelaskan perbedaan:

Clustered Index

Non-Clustered Index

Soal 4

Jelaskan perbedaan:

Dense Index

Sparse Index

Soal 5

Mengapa B+ Tree lebih banyak digunakan dibanding struktur index lainnya?

Soal 6

Jelaskan kelebihan dan kekurangan Hash Index.

Soal 7

Apa yang dimaksud dengan:

Full Table Scan

dan kapan hal tersebut terjadi?

Soal 8

Jelaskan tahapan:

Parsing

Optimization

Execution

dalam Query Processing.

Soal 9

Apa fungsi Query Optimizer pada DBMS?

Soal 10

Analisis dampak jika sebuah tabel berisi jutaan data tetapi tidak memiliki index sama sekali.

BAB 9 selesai.

Selanjutnya: KUIS 2 (Bab 7–9).